

Security in Informatics and Communications

Guide #6:

Digital Signatures

Symmetric vs. Asymmetric

Symmetric

One key for everything

All participants know the same information.

Fast and simple but has drawbacks

- Involved parties need to share the same key
- Needs a secure channel to share keys (can be hard)
- Keys do not distinguish any involved parties
- Distribution does not scale

Asymmetric

A key pair for different operations

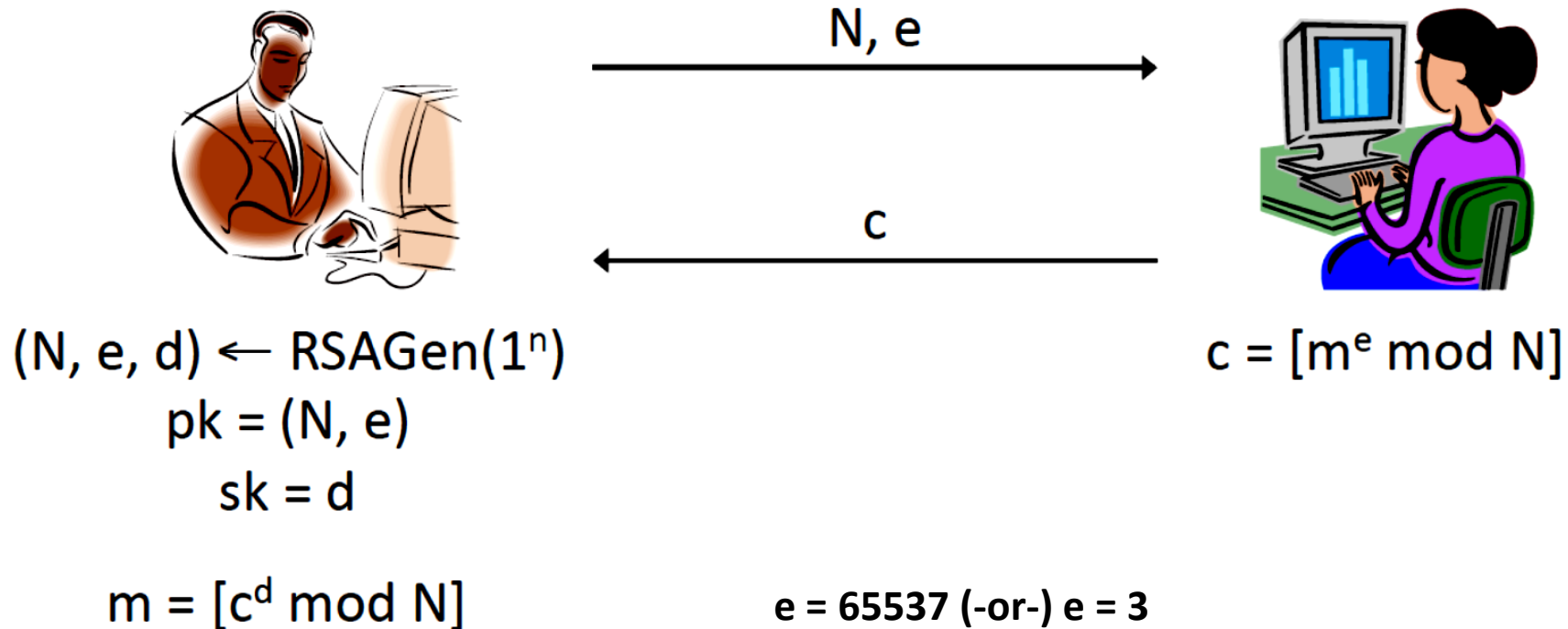
Different participants have different material

Very secure but has drawbacks

- Slow and complex
- Computationally intensive (expensive)

Asymmetric ciphers (*RSA n-bits*)

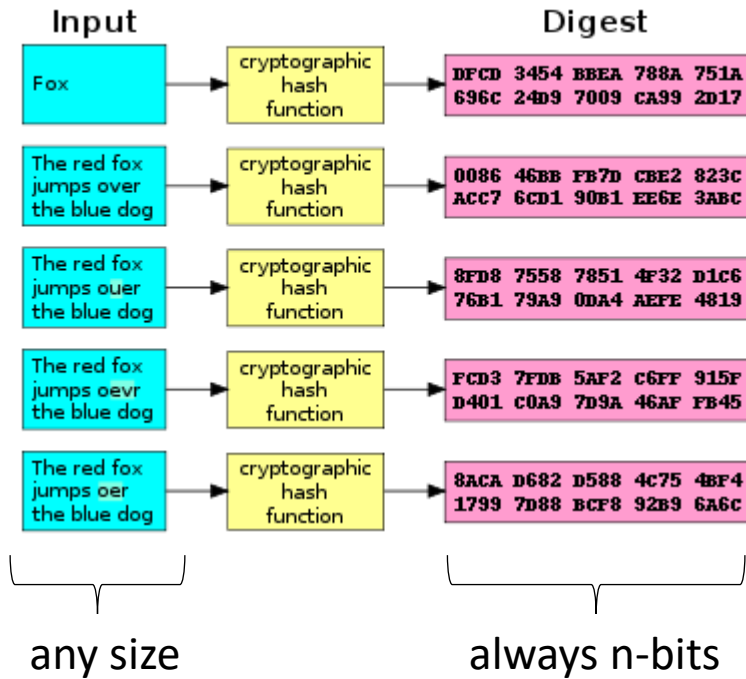
“Plain” RSA encryption



Practical Use

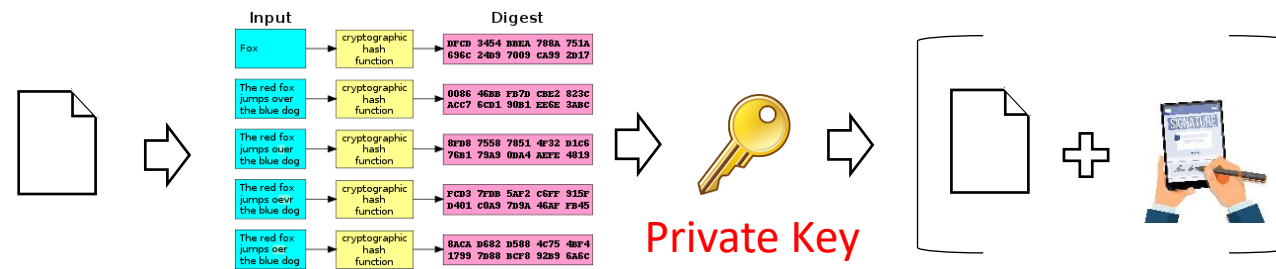
- Confidentiality (e.g., File Encryption)
 - Encrypt data with ***public key***
 - Decrypt with ***private key***
 - Source is not authenticated!
-  - Authenticity (e.g., Digital Signatures, Authentication)
 - Encrypt challenge/identifier with ***private key***
 - Decrypt with ***public key***
 - Source is now authenticated!

Recap : Digest Function (aka “Cryptographic Hash”)

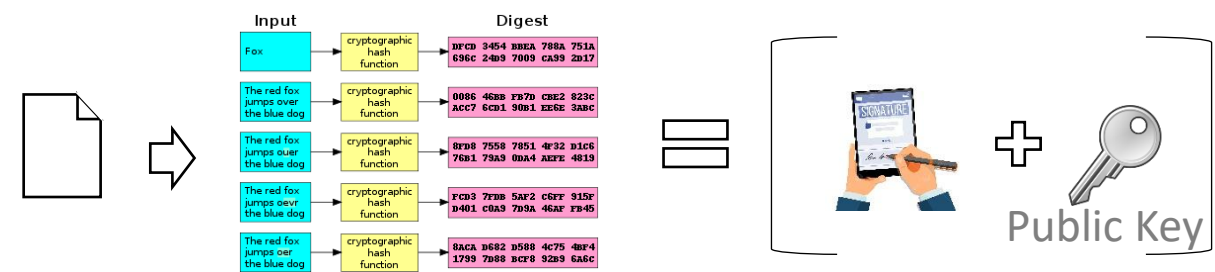


- ▶ Give a fixed-length value from a variable-length text
 - ◆ Sort of text “fingerprint”
- ▶ Produce very different values for similar texts
 - ◆ Cryptographic one-way hash functions
- ▶ Relevant properties:
 - ◆ Preimage resistance
 - Given a digest, it is infeasible to find an original text producing it
 - ◆ 2nd-preimage resistance
 - Given a text, it is infeasible to find another one with the same digest
 - ◆ Collision resistance
 - It is infeasible to find any two texts with the same digest
 - Birthday paradox

Sign a File / Data



Verify signature



In practice

- These operations are already encapsulated into methods:

- sign(...)

```
>>> from cryptography.hazmat.primitives import hashes
>>> from cryptography.hazmat.primitives.asymmetric import padding
>>> message = b"A message I want to sign"
>>> signature = private_key.sign(
...     message,
...     padding.PSS(
...         mgf=padding.MGF1(hashes.SHA256()),
...         salt_length=padding.PSS.MAX_LENGTH
...     ),
...     hashes.SHA256()
... )
```

- verify(...)

```
>>> public_key = private_key.public_key()
>>> public_key.verify(
...     signature,
...     message,
...     padding.PSS(
...         mgf=padding.MGF1(hashes.SHA256()),
...         salt_length=padding.PSS.MAX_LENGTH
...     ),
...     hashes.SHA256()
... )
```


Any Questions?